

Amazing Hand 小白复刻手册（Linux + Python 零依赖路线）

写给第一次做这个项目的新手，手把手从买料到让整只手动起来。

本手册基于一次真实复刻全过程整理，把每条命令、预期结果、以及踩过的每个坑都写进来了。

适用：右手 + Linux 电脑 + Waveshare 串口总线驱动 + Python。

用仓库里零依赖脚本（`AmazingHand_SetID.py` / `AmazingHand_Calib.py` /

`AmazingHand_Demo_NoDeps.py`），不用 Windows 的 FD 软件，也不用 rustypot。

⚠ 开跑前必改的两处（每台机器都不一样！）

clone 下来不能直接跑，有两个地方必须换成你自己的，否则连不上或动作乱套：

① 串口路径（换成你自己驱动板的序列号）

脚本里默认的 `--port` 是作者机器的 by-id 路径（`...usb-1a86_USB_Single_Serial_5B61034381-if00`），

你的序列号不同。先跑下面两行得到你自己的，再用 `--port "$PORT"` 传入（见 5.2）：

```
PORT=$(ls /dev/serial/by-id/ | grep -i Single_Serial | head -1);
PORT="/dev/serial/by-id/$PORT"

echo "$PORT"      # 这就是你要用的串口
```

② 标定值（填你自己实测的）

AmazingHand_Demo_NoDeps.py 顶部 FINGERS 里的 off（中位偏移）和 flip（方向）

是作者手的标定结果，**你必须按自己第 7 节实测的值填**（见 8）。

方向装反/偏移不对会导致动作乱套。

这两处不改，轻则连不上串口，重则手指往反方向使劲顶坏机构。**务必先改再跑。**

目录

- 0. 这个项目是什么
- 1. 准备材料（BOM）
- 2. 3D 打印
- 3. 机械装配总览
- 4. 接线与供电
- 5. Linux 软件环境
- 6. 第一步：设置 8 个舵机 ID
- 7. 第二步：中位标定
- 8. 第三步：跑整手 Demo
- 9. 常见问题排查（含真实现象）
- 10. 命令速查表
- 11. 文件清单
- 12. 拍教程的建议

0. 这个项目是什么

Amazing Hand 是一只开源仿人机械手：**4 根手指、8 个自由度**，每根手指由

2 个 Feetech SCS0009 小舵机并联驱动，全 3D 打印，舵机全藏在手内、无外部线缆，成本 < 200€。

每根手指 2 个舵机如何合成动作（理解这点，标定和写动作就通了）：

- 屈/伸（开合）：两舵机反向转 → 手指握拢 / 张开
- 外展/内收（左右摆）：两舵机同向转 → 手指左右摆

一个舵机负责"右半边"，一个负责"左半边"，两者配合就能做出 2 自由度动作。

1. 准备材料（BOM）

完整清单见仓库 README 的 BOM 链接。核心：

类别	物件	数量
舵机	Feetech SCS0009	8
舵机配件	十字摇臂、舵机螺丝 2x7 / M2x4	配套
传动	球头拉杆、连杆、销钉	见 BOM
结构	3D 打印件（手指框架、外壳、手掌、手腕接口）	STL 在 cad/
控制板	Waveshare 串口总线舵机驱动板 （USB→Feetech 总线）	1
电源	外部 5V / ≥2A DC 适配器（必备）	1
线材	Feetech 三针总线连接线（菊花链用）	若干

也可以用 Arduino + Feetech TTL Linker，但本手册走 Waveshare + Python 路线。

2. 3D 打印

参考 docs/AmazingHand_3D 打印技巧_中文.md。要点：

- 右手件以 R 开头，左手件以 L 开头；**手指本身左右通用**，只有部分件对称。
- 柔性外壳建议 **TPU**；框架用 PLA / PETG。
- 打印件不完美会带来角度误差——这正是后面要"标定"的原因。

3. 机械装配总览

跟着 docs/AmazingHand_装配指南_中文.md 一步步装。**关键的装配顺序：**

1. 先做好球头拉杆（用专用工装定长）、摇臂总成。
2. 把 2 个舵机固定进手指框架，接好连杆/球头拉杆。
3. **摇臂先别拧死到舵机轴上**——要等软件把舵机转到 0°（中位）后再装摇臂对齐（见第 7 节）。这是保证中位准确的诀窍。

建议：**边装边标定**。每装好一根手指就走一遍"设 ID + 标中位"，
比全部装完再统一调更省心。

4. 接线与供电

电脑 USB —— Waveshare 驱动板 —— 舵机 A —— 舵机 B —— 舵机 C ...（菊花链）

└—— 外部电源(5V) 接驱动板电源端子

1. Waveshare 驱动板 → 电脑 USB。
2. **外部电源**接驱动板电源端子（注意正负极）。
3. 舵机用三针总线线**菊花链**串接：每个舵机有两个并排同款插座，一进一出。
4. 波特率统一 **1000000 (1M)**。

⚠ **本项目最常见的故障来源：供电/接线。** 实测中外部电源接头、USB 接头多次松动，导致"扫描突然全部消失"。**每个接头都插到底**，电源要稳。

⚠ 任何一个接头虚接，会让**整条总线**通信失败（不只是那一个舵机）。

⚠ 没接外部电源，舵机完全不响应。

5. Linux 软件环境（零依赖）

本路线**只需系统自带的 stty + Python3 标准库**。设 ID / 标定 / Demo 全程**不用 pip、不用第三方库**。

5.1 串口读写权限（一次性）

```
sudo usermod -aG dialout $USER    # 加入 dialout 组
```

```
# 然后【注销重新登录】（或重启）让权限生效
```

5.2 找到驱动板的串口（重要）

插上驱动板，看串口设备：

```
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

```
ls -l /dev/serial/by-id/
```

不同芯片枚举不同：

- CH340 芯片 → `/dev/ttyUSB*`

- CH343 / CH9102 芯片 → 可能是 `/dev/ttyACM*`

强烈建议用 `/dev/serial/by-id/usb-...` 那个带序列号的固定路径，

它重启、插拔都不变；而 `ttyACM0/1` 这种编号在插了多个 USB 设备时会乱跳。

自动找回驱动板路径的小技巧（CH343 板，厂商串 `Single_Serial`）：

```
PORT=$(ls /dev/serial/by-id/ | grep -i Single_Serial | head -1)
```

```
PORT="/dev/serial/by-id/$PORT"
```

```
echo "$PORT"
```

下文统一用 `$PORT` 代指你的驱动板。每开一个新终端先跑上面两行设好它。

（脚本里 `--port` 默认值也已写成一个 `by-id` 路径，但**那是作者机器的序列号，**

你必须换成自己的，所以推荐每次显式传 `--port "$PORT"`。）

5.3 验证通信

接一个舵机、上电，扫描：

```
python3 AmazingHand_SetID.py --port "$PORT" --scan
```

看到 找到的 ID: [1] 就通了（新舵机出厂都是 ID=1）。扫不到 → 看第 9 节。

6. 第一步：设置 8 个舵机 ID

整只手 8 个舵机要有**唯一 ID**，约定如下：

手指	ID	备注
食指 Index	1,	奇数=右侧舵机，偶数=左侧舵机
	2	
中指 Middle	3,	(按装配指南视角)
	4	
无名指 Ring	5,	
	6	
拇指 Thumb	7,	
	8	

6.1 三条铁律（血泪换来的）

铁律 1：一次只接一个新舵机。

新舵机出厂全是 ID=1。如果总线上已经有别的 ID=1（比如已装好的食指 1 号），你发"把 ID=1 改成 X"的指令会**同时改掉所有 ID=1 的舵机！**
(实测真把食指 1 号误改成了 5，后来才修回来。)

铁律 2：改 ID 必须加 --unlock。

不加的话 ID 看似改了，**一断电就退回 1**（写不进 EEPROM）。

铁律 3：改之前先扫描确认总线上"只有一个 ID=1"。

6.2 安全设 ID 的标准流程

只接当前这一个新舵机（其余手指全部拔掉），上电，然后：

1) 先扫描，必须只看到 [1]

```
python3 AmazingHand_SetID.py --port "$PORT" --scan
```

2) 确认只有一个 [1] 后，改成目标 ID（这里以 2 为例），务必带 --unlock

```
python3 AmazingHand_SetID.py --port "$PORT" --old 1 --new 2 --unlock
```

3) 断电再上电，扫描确认存住了

```
python3 AmazingHand_SetID.py --port "$PORT" --scan      # 应显示 [2]
```

带安全检查的一条龙（推荐，扫到不是单个 [1] 就拒绝执行）：


```
IDS=$(python3 AmazingHand_SetID.py --port "$PORT" --scan 2>&1 | grep -o "\
[.*\\]")

if [ "$IDS" = "[1]" ]; then

    python3 AmazingHand_SetID.py --port "$PORT" --old 1 --new 2 --unlock

else

    echo "    停！总线不是单独一个 ID=1（实际 $IDS），先把其余舵机拔掉。"

fi
```

依次把 8 个舵机设成 1~8。每设一个，**换下一个新舵机前先断电**。

成功输出长这样：

```
确认到 ID=1 解锁 EEPROM ... 写入新 ID: 1 -> 2 重新上锁 EEPROM ... ✓ 成功：舵机现在的 ID =
2
```

6.3 万一误改了怎么办（救援）

如果不小心把某个已设好的舵机也改了（比如食指 1 号变成了 5）：

1. 把那根受影响的手指**单独**接上（不要接同 ID 的新舵机）；
2. 此时总线上那个错误 ID 只有一个，可安全改回：

```
bash python3 AmazingHand_SetID.py --port "$PORT" --old 5 --new 1 --unlock
```

7. 第二步：中位标定（逐指）

舵机摇臂和舵机轴是**花键咬合**，每次装角度都差几度，所以每个舵机要标一个**中位偏移**。用 `AmazingHand_Calib.py`（零依赖，不用 `rustypot`）。

7.1 标定原理（一句话）

软件先把舵机转到 **0°（中位）** → **趁机装摇臂对齐** → **跑闭合测试，看是否对称，不齐就在软件里加几度偏移补偿。**

Calib 工具的动作定义（角度相对各自舵机中位）：

命令	ID1	ID2	说明
---	---	---	---
middle	0°	0°	中位（装摇臂用）
close	+90°	-90°	闭合（两舵机反向）
open	-30°	+30°	张开
cycle	—	—	开合循环（看效果/解除堵转）
read	—	—	读当前角度
relax	—	—	松扭矩（可手扳）

7.2 逐指标定走查（以食指 ID 1、2 为例）

① 让舵机到中位，再装摇臂

```
python3 AmazingHand_Calib.py middle --ids 1 2 --port "$PORT"
```

舵机转到 0° 并**自保持**（程序退出也保持，因为扭矩已使能）。

提示：如果刚上电舵机离 0° 很远，第一次可能显示停在 $\pm 60^\circ$ 左右——那只是“还没转到位”的瞬时读数，等一两秒再 `read` 一次通常就到 0° 了。

趁舵机停在 0° ，把两个摇臂按"中位"姿态**尽量对齐**套上舵机轴，拧 M2x4 固定。

② 确定开合方向（左右手 / 拇指会不同！）

```
python3 AmazingHand_Calib.py close --ids 1 2 --port "$PORT"
```

看手指实际动向：

- 若**真往握拢方向闭合** → 方向对，这根**不加** `--flip`。

- 若**动反了**（该闭合却张开）→ 这根以后所有命令都加 `--flip`：

```
bash python3 AmazingHand_Calib.py close --ids 1 2 --flip --port "$PORT"
```

本右手实测结论：

- 食指、中指、无名指 → **需要** `--flip`

- **拇指** → **不需要** `--flip`（拇指安装朝向和其它三指相反，属正常）

闭合时舵机命令 $\pm 90^\circ$ ，实际常只到 $\pm 72^\circ$ —— 因为手指**碰到机械闭合限位**了，

两边对称地差一样多就说明左右平衡，是正常的。

③ 看对称、必要时微调

- 闭合对称、到位 → 偏移就是 0/0（实测四根手指都是 0/0，运气好/摇臂装得准）。

- 偏向某侧 → 给偏的那个舵机加几度，反复试：

```
bash python3 AmazingHand_Calib.py close --ids 1 2 --flip --off1 3 --off2 0 --port "$PORT"
```

⚠ 闭合时舵机**顶住限位会堵转发热**，别长时间停在闭合位。看完赶紧

cycle 或 middle，或用 relax 松开。

④ 看整体开合 + 记录

```
python3 AmazingHand_Calib.py cycle --ids 1 2 --flip --off1 0 --off2 0 --n 3 --
```

```
port "$PORT"
```

把这根手指的 flip / off1 / off2 记到 PythonExample/calibration.txt。

四根手指各重复 ①~④。 别忘拇指那根**不带 --flip**。

7.3 实测标定结果（参考）

手指	id s	fli p	off 1	off 2
食指 index	1, 2	是	0	0
中指 middle	3, 4	是	0	0
无名 ring	5, 6	是	0	0
拇指 thumb	7, 8	否	0	0

你的 off 值可能不是 0，取决于摇臂装得多准——以你实测为准。

8. 第三步：跑整手 Demo

编辑 AmazingHand_Demo_NoDeps.py 顶部的 FINGERS，把已标定的手指设

present=True 并填 off、flip（**拇指 flip=False**）：

```
FINGERS = {
```

```
"index": dict(ids=(1, 2), off=(0.0, 0.0), flip=True, present=True),

"middle": dict(ids=(3, 4), off=(0.0, 0.0), flip=True, present=True),

"ring": dict(ids=(5, 6), off=(0.0, 0.0), flip=True, present=True),

"thumb": dict(ids=(7, 8), off=(0.0, 0.0), flip=False, present=True),

}
```

把 8 个舵机全接总线、上电，先确认全在线：

```
python3 AmazingHand_SetID.py --port "$PORT" --scan    # 应为
[1,2,3,4,5,6,7,8]
```

跑：

```
python3 AmazingHand_Demo_NoDeps.py --port "$PORT"      # 跑一遍

python3 AmazingHand_Demo_NoDeps.py --port "$PORT" --loop  # 循环（拍视频），
Ctrl-C 停

python3 AmazingHand_Demo_NoDeps.py --port "$PORT" --relax  # 结束松扭矩
```

动作含：醒手、招手、左右摆、慢握拳、快速点按、画圈、收势。

只装了部分手指也能跑，它只动 `present=True` 的手指。

多指必须同步驱动：Demo 内部用 **SYNC WRITE（广播一帧同时驱动所有舵机）**。

如果改成逐个发指令，4 根手指会明显**卡顿/不同步**。这是实测验证过的关键点。

9. 常见问题排查（含真实现象）

现象	原因 / 处理
<code>--scan</code> 一个都扫不到	最常见是供电掉了或接头虚接 。先确认外部电源插着、5V 有输出；再逐个检查菊花链接头。换串口路径试（ <code>ttyUSB*</code> ↔ <code>ttyACM*</code> ）。
接一个能扫到，接两个就都没了	两个舵机 ID 冲突 （都是 1），或它们之间那段总线线虚接/插反。
扫描中途突然全没了	操作时 碰松了电源或 USB 。本项目实测多次发生。插紧再扫。
改了 ID 断电后又变回 1	改 ID 时 没加 <code>--unlock</code> 。加上重改。
改 ID 把别的舵机也改了	总线上有 多个 ID=1 时改 ID 会全改。务必"一次只接一个新舵机"。按 6.3 救援。
官方 *.py（rustypot）报 Parsing error	rustypot 在某些 CH343 板上不兼容。 改用本手册的零依赖脚本 。
闭合方向反了	加 <code>--flip</code> （或拇指那种本来就不加 flip 的，反过来去掉）。
命令 ±90° 实际只到 ±72°	正常：手指顶到机械闭合限位。两边对称即可。
舵机发烫	长时间堵转（顶限位）。别久停在闭合位，及时 <code>cycle/middle/relax</code> 。
多指 Demo 卡顿/不同步	必须用 SYNC WRITE（本 Demo 已用）。

现象

原因 / 处理

串口权限不足

`sudo usermod -aG dialout $USER` 后**重新登录**。

`stty: ...: 没有那个文件`

驱动板 USB 掉了/路径变了。重插，用 5.2 的自动找回命令。

或目录

10. 命令速查表

设串口变量（每个新终端先跑）

```
PORT=$(ls /dev/serial/by-id/ | grep -i Single_Serial | head -1);
```

```
PORT="/dev/serial/by-id/$PORT"
```

—— 设 ID ——

```
python3 AmazingHand_SetID.py --port "$PORT" --scan # 扫描
```

```
python3 AmazingHand_SetID.py --port "$PORT" --old 1 --new N --unlock # 改 ID（带解锁）
```

—— 标定 —— (flip 视手指而定; 拇指不加 flip)

python3 AmazingHand_Calib.py middle --ids A B --port "\$PORT" # 到中位, 装摇臂

python3 AmazingHand_Calib.py close --ids A B --flip --port "\$PORT" # 闭合, 看对齐

python3 AmazingHand_Calib.py cycle --ids A B --flip --n 3 --port "\$PORT" # 开合循环

python3 AmazingHand_Calib.py read --ids A B --port "\$PORT" # 读角度

python3 AmazingHand_Calib.py relax --ids A B --port "\$PORT" # 松扭矩

可选: --off1 X --off2 Y 微调中位; --speed S 调速度(0=最大)

—— Demo ——


```
python3 AmazingHand_Demo_NoDeps.py --port "$PORT"          # 跑一遍

python3 AmazingHand_Demo_NoDeps.py --port "$PORT" --loop    # 循环

python3 AmazingHand_Demo_NoDeps.py --port "$PORT" --relax    # 结束松扭矩
```

11. 文件清单

文件	作用
PythonExample/AmazingHand_SetID.py	扫描 / 设置舵机 ID（零依赖）
PythonExample/AmazingHand_Calib.py	中位标定 middle/close/open/cycle/read/relax（零依赖，含 SYNC WRITE）
PythonExample/ AmazingHand_Demo_NoDeps.py	整手花样 Demo（零依赖，可扩展 4 指）
PythonExample/calibration.txt	8 个舵机的标定值记录
docs/AmazingHand_装配指南_中文.md	机械装配图文步骤
docs/AmazingHand_3D 打印技巧_中文.md	3D 打印参数建议

12. 拍教程的建议

把这份手册当脚本，建议按这个顺序录制，每段都能独立成节：

1. **开箱与材料**（第 1 节）：展示 8 个舵机、驱动板、电源、打印件。

2. **接线讲解**（第 4 节）：重点演示菊花链和"接头要插牢"——可以故意演示一次"接头松了→扫不到"再插好，观众最容易踩这个坑。
3. **软件环境**（第 5 节）：演示 `--scan` 第一次通信成功的那一刻。
4. **设 ID**（第 6 节）：**重点强调三条铁律**，演示一次"安全检查脚本"。
可以讲那个"误改食指"的真实故事当反面教材。
5. **标定**（第 7 节）：拍"舵机转到 0°→装摇臂→闭合检查对称"的全过程，
特写摇臂对齐。强调**拇指不加 flip** 这个反直觉点。
6. **整手 Demo**（第 8 节）：高潮——整手动起来。讲一下 SYNC WRITE 为什么重要
（对比卡顿 vs 同步）。
7. **排错**（第 9 节）：单独做一节"翻车合集"，照着表演示每种现象。

录制 Demo 视频用 `--loop` 连续跑，方便从任意角度补拍。

祝复刻顺利，也祝你的教程大火